

一个网络请求的一生

从输入 URL 到收到网页内容

408 · NET-I01

这本综合册要解决什么问题

浏览器里只输入了一个 URL，但网络不会因此机械执行一遍“DHCP-DNS-ARP-TCP-HTTP”。主机可能已经有网络配置，DNS、ARP、TCP 或 HTTP 缓存也可能已经存在；不同链路还会使用不同的帧格式。

本册要回答的是：在当前已有状态和当前网络拓扑下，为了完成这次访问，下一步真正需要做什么；每产生一条数据报，它又怎样一跳一跳到达目标。

本册负责组合，不重复讲协议细节

NET01-NET08 分别负责时延、单跳交付、可靠传输、IP 转发、路由、TCP/UDP、拥塞控制和应用层协议；NET-B01-B06 负责这些专题之间的稳定接口。

本册只把它们接成一个完整过程：什么状态可以直接复用，什么状态必须先建立；一条数据报经过主机、AP、交换机、路由器和不同链路时，哪些字段保持、哪些字段改变；失败时第一处断点在哪里。

1 先建立正确的总图：请求推进和逐跳交付是两个不同过程

一个容易产生的错误画法是：

DHCP → DNS → ARP → TCP → HTTP.

这条线只能当知识点清单，不能当真实执行顺序。原因很直接：DNS 查询本身就要先被送到 DNS 服务器；TCP 的 SYN、ACK 和 HTTP 数据也都要经过 IP 转发和当前链路交付。ARP 只在需要解析当前 Ethernet 下一跳时出现，Native PPP 链路根本不需要 Ethernet MAC。

因此把全过程分成两个独立又协同的维度（如图 1 所示）：

1. **请求推进（事务层）**：这次访问还缺什么状态？已有状态直接复用，只补真正缺失的部分。
2. **逐跳交付（数据报层）**：一旦已经形成一条普通 IP 数据报，当前节点怎样根据自己的 FIB 选出下一跳，并用当前链路把它送过去？

1.1 请求推进：已有状态决定哪些步骤根本不会发生

如果浏览器的 HTTP 缓存已经足够满足当前请求，网络访问可以整体消失。若必须联网，再依次检查：

- 主机有没有可用的 IP 地址、前缀、默认网关和 DNS 服务器配置；
- URL 中的主机名是否已经对应到可用的目标 IP；
- 主场景需要 TCP 时，是否已经有可复用的连接；
- 最后才是实际的 HTTP 请求和响应。

这里的“依次检查”表示前置条件关系，不表示每次都重新执行这些协议。

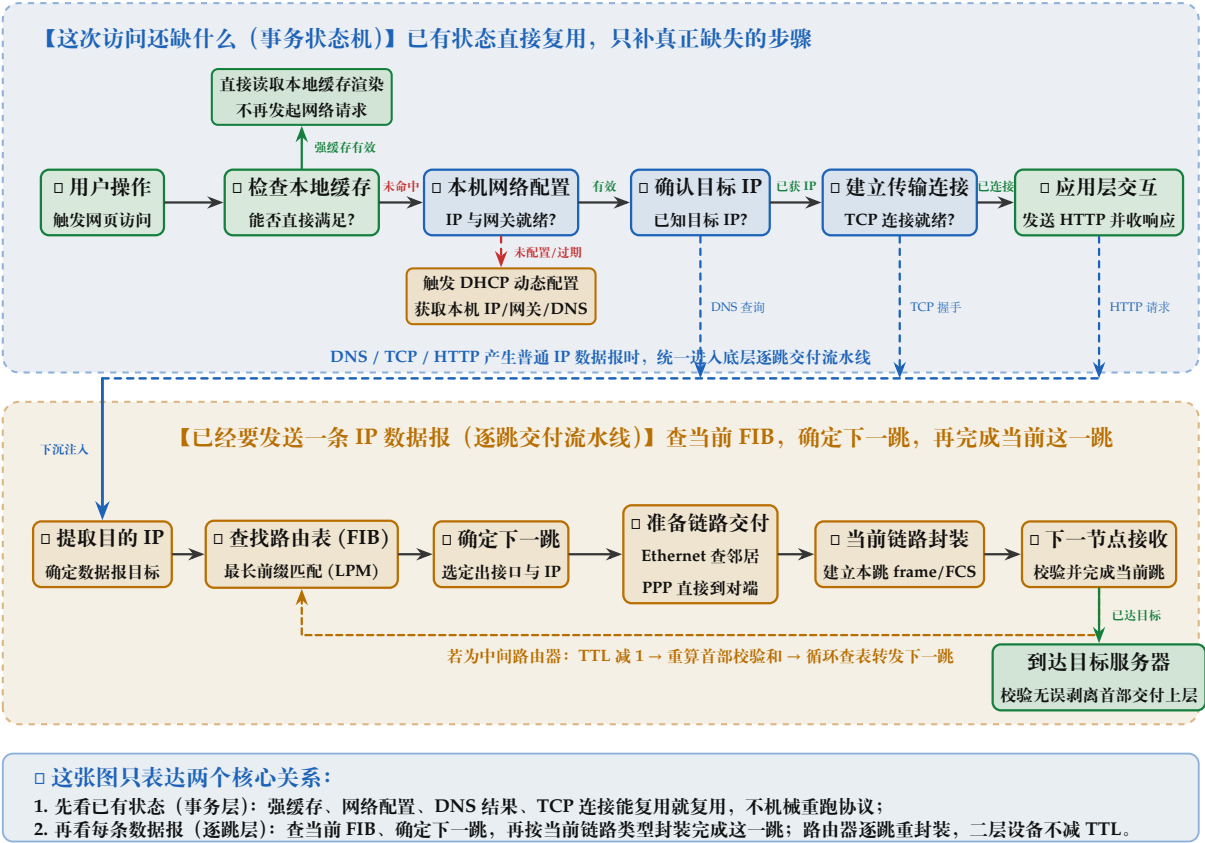


图 1 请求推进与逐跳数据报双层执行模型。上方事务状态机按需触发报文下沉，下方流水线统一处理每条普通 IP 数据报的逐跳交付。

1.2 逐跳交付：当前节点只负责把数据报送到下一节点

对于一条普通 IP 数据报，当前主机或路由器真正执行的是：

目的 IP $\xrightarrow{\text{查当前 FIB}}$ (出接口, 下一跳) $\xrightarrow{\text{准备本跳链路交付}}$ 下一节点.

“准备本跳链路交付”不是固定等于 ARP：

- Ethernet / 802.11 场景中，若缺少当前下一跳的链路地址，可能需要 ARP 或 ND；
- 点对点 PPP 只有一个对端，不需要 Ethernet 式 MAC 查询；
- 换到另一种链路时，使用该链路自己的封装和寻址方式。

若下一节点仍是路由器，入链路帧在该路由器处结束；IPv4 转发时路由器处理 TTL，再用自己的 FIB 重新决定下一跳。这个过程不断重复，直到目的主机接收数据报。

2 看综合题时先问四个具体问题

不需要先给题目贴“这是 DNS 题还是 TCP 题”的标签。先回答下面四问，通常就能知道当前应该调用哪个专题。

先问什么	要看清什么	例子
现在处理的对象是什么？	应用层报文、TCP 报文段、IP 数据报、链路帧还是物理信号	HTTP 状态码不能拿来解释路由器转发

这个机制影响到哪里？	当前链路、广播域、一个路由跳、通信两端、一个 AS 或某个应用服务	ARP 不跨路由器；TCP 连接由两端维护
哪台设备或哪个协议模块保存状态？	主机、交换机、路由器、路由协议进程、DNS 解析器、TCP 端点等	MAC 表在交换机；FIB 在路由器；cwnd 在发送端
什么事件会让状态改变？	报文到达、ACK、超时、缓存过期、路由更新、链路变化等	新 LSA 会改变路由知识；ACK 会改变 TCP 发送端状态

控制面与数据面的区别也从这四问自然得到：OSPF/BGP 负责形成和更新路由状态；普通数据报到来时，路由器只使用当前已经安装的 FIB，不会为了这一个包临时重新运行路由协议。

3 贯穿母场景：先把真实拓扑画对

主场景固定为 IPv4 + HTTP over TCP（全链路拓扑结构如图 2 所示）。互相替代的技术不强行塞进同一条真实路径，而放到后面的替换场景中比较。

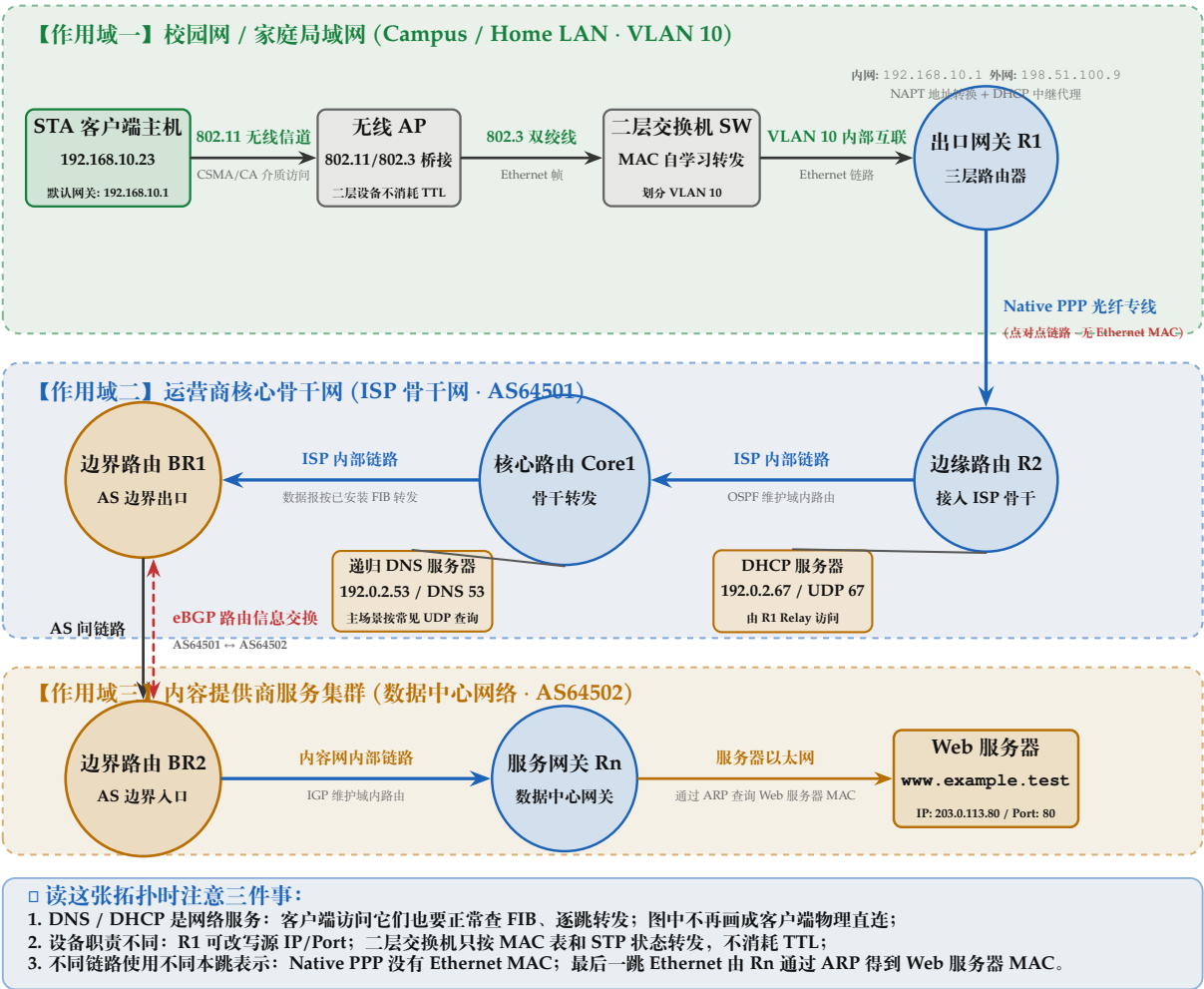


图 2 贯穿母场景拓扑结构。全流程跨越局域接入、骨干路由与数据中心三大作用域；实线表示实际数据转发链路，虚线表示 eBGP 控制关系。

3.1 固定示例地址

对象	示例地址 / 作用
客户端	192.168.10.23/24
R1 的 LAN 接口	192.168.10.1/24，作为默认网关
R1-R2 点对点 WAN	198.51.100.8/30；R1 为 198.51.100.9，R2 为 198.51.100.10
递归 DNS 服务器	192.0.2.53:53
DHCP 服务器	192.0.2.67:67
服务器 LAN 网关 Rn	203.0.113.1/24
Web 服务器	203.0.113.80:80，域名 www.example.test

这些地址只为了让后面的字段变化可追踪，不表示真实 Internet 地址分配。

3.2 这张图应该怎样读

主拓扑只回答“设备在哪里、链路是什么、哪些设备属于同一个作用域”。横向或纵向距离不表示真实距离和时延。

- STA、AP、S1、S2 和 R1 的 LAN 接口处在接入网络。S1-S2 有冗余链路，STP 只让其中一条进入普通转发路径。
- R1-R2 是 Native PPP 点对点链路，所以这一段没有 Ethernet 源/目的 MAC。
- AS64501 内由 OSPF 等域内路由机制维护路由状态；BR1 与 BR2 之间通过 eBGP 交换跨 AS 的前缀和路径属性。
- DNS 和 DHCP 服务器挂在 ISP 可达的服务网段。客户端访问它们仍然要经过普通 IP 转发；图中没有画“客户端直连 DNS”的假链路。

3.3 路由表从图里拆出来单独读

拓扑图不应该同时塞入大块转发表。真正转发时，每台设备只读自己的当前 FIB。例如：

设备	典型 FIB 状态	当前数据报怎么走
客户端	192.168.10.0/24 为直连网段；默认路由经 192.168.10.1	访问远端 Web 时，IP 目的地址仍是 Web 服务器，但下一跳是 R1
R1	LAN 直连；198.51.100.8/30 直连；其余前缀经上游 R2	LPM 先确定 PPP 出接口，再做该链路的封装
Rn	203.0.113.0/24 直连；其余前缀经上游	访问 203.0.113.80 时，下一跳终于就是最终目的主机

若题目再加入 /32、聚合前缀和默认路由，仍然只做最长前缀匹配：匹配成功的候选中，前缀越长越具体。

4 同一个 URL 为什么可能产生完全不同的报文

真正的输入不只有 URL，还包括当前已经保存的状态：

```

HostConfig    = 有效 / 缺失 / 过期
DNSCache      = 命中 / 未命中 / 过期
InstalledFIB  = 可用 / 缺路由 / 旧版本
Neighbor      = 命中 / 未命中 / 正在解析
SwitchState   = 已学习 / 目的未知 / STP 阻塞
NATState      = 已有映射 / 无映射
TCPState      = 可复用 / 不存在 / 已复位 / 正在关闭
HTTPState     = 可直接使用缓存 / 需重新验证 / 未命中
  
```

每个状态都有自己的匹配条件、作用范围和有效时间。“刚访问过一次”不等于所有状态都一定有效。状态剪裁的判定决策流如图 3 所示。

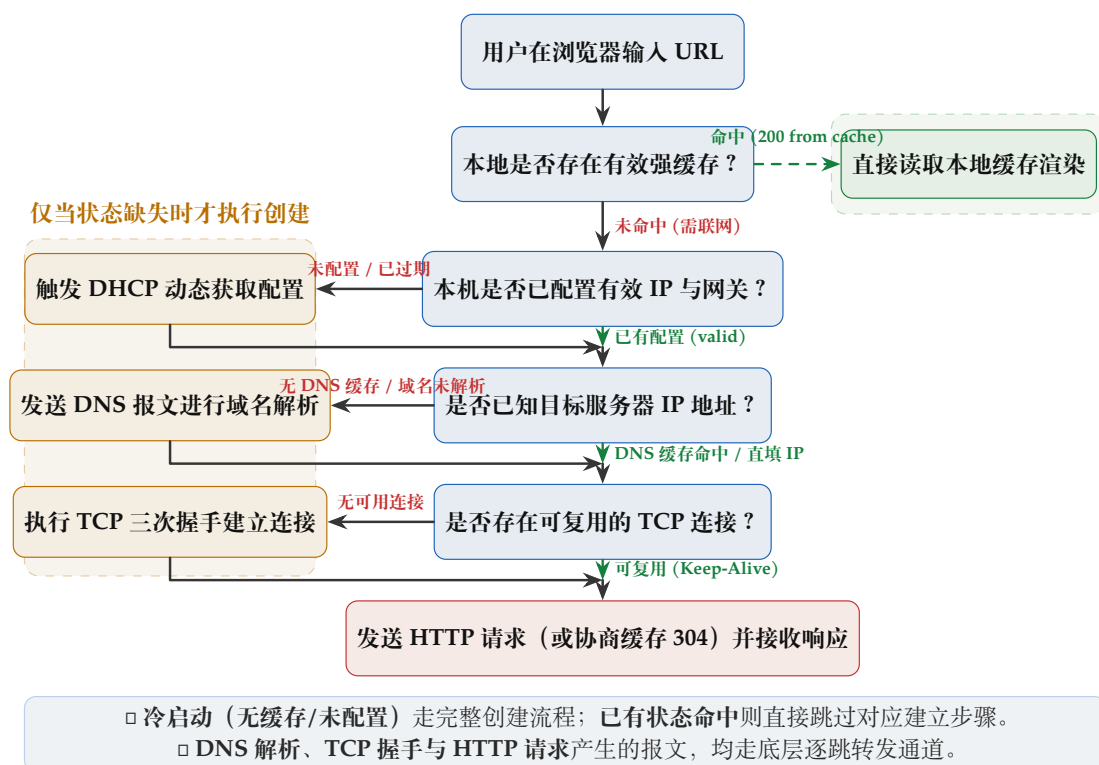


图 3 状态命中与网络事件剪裁 DAG。已有配置、DNS 结果和 TCP 连接可直接复用；各阶段产生的报文均走底层逐跳转发通道。

这张图还有一个重要用途：它阻止我们把“第二次访问”简单写成“什么都不用做”。例如 DNS 缓存命中只说明不用重新解析名字；它并不能自动保证 TCP 连接仍可复用，也不能保证 ARP 表项还在，更不能保证 HTTP 缓存一定新鲜。

5 开始普通通信前：没有网络配置时先完成 DHCP

如果主机已经有有效配置，本节直接跳过。若没有，IPv4 主场景先使用 DHCP 获得：

(主机 IP/前缀, 默认网关, DNS 服务器, 租约有效期)。

客户端刚启动时还没有普通单播源地址。经典 DORA 过程是 Discover、Offer、Request、ACK；初始报文可以使用源地址 0.0.0.0 和受限广播 255.255.255.255，客户端使用 UDP 68，服务器使用 UDP 67。

若 DHCP 服务器不在本子网，路由器不会直接转发原始二层广播。R1 上的 DHCP Relay（中继）先接收本地 DHCP 消息，再把请求按普通 IP 路由送到远端服务器。这个例子很好地说明：**应用层协议可以用来建立网络层配置，但协议所在层和它最终修改的状态不必相同。**

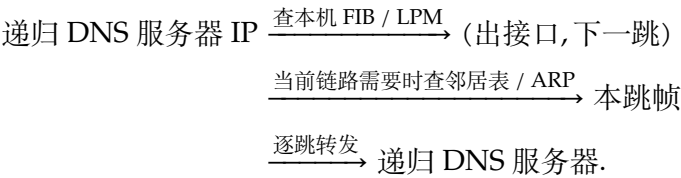
IPv6 不照搬这条路径。常见主机可通过 Router Advertisement / SLAAC 获得地址和前缀相关信息，并用 Neighbor Discovery 完成邻居发现；ND 使用 ICMPv6 multicast，不是“IPv6 版 ARP 广播”。

6 DNS：先判断是否需要解析名字，再把查询报文真正送出去

浏览器先从 URL 中取出 host（主机部分）。再分三种情况，不要一看到 URL 就默认执行 DNS：

当前情况	这一步实际发生什么
host 本身就是 IP 地址	跳过 DNS，直接把这个 IP 作为后续通信的目标地址
DNS 缓存中有仍可使用的记录	复用已有结果，不重新发送 DNS 查询
host 是主机名，而且没有可用缓存	创建一次 DNS 查询，先把主机名解析成目标 IP

当 DNS 缓存未命中时，客户端已经知道递归 DNS 服务器的 IP，例如 192.0.2.53。但“知道 DNS 服务器在哪”并不等于查询已经到达它。DNS 查询报文本身也必须先被发送出去，客户端仍要完成：



因此一个非常重要的顺序是：**ARP/ND 完全可能发生在目标网站的 DNS 应答之前**。这里 ARP/ND 解析的是“当前 DNS 查询报文的下一跳”，通常是默认网关；它不是在询问 www.example.test 对应 Web server 的 MAC。

递归 DNS 服务器收到查询后，如果自己的缓存也没有答案，它再从自己的网络环境出发，按需要查询根 DNS、顶级域 DNS（TLD）和权威 DNS 服务器。这里发生的是**递归 DNS 服务器自己的后续查询**；不能把客户端画成亲自依次与这些服务器通信。

DNS 最终交给客户端的是名字解析结果，例如一个或多个 A/AAAA 地址及相应 TTL：



它不输出 next-hop MAC，也不保证这个 Web server 此刻一定可达。CNAME、NS、MX 等记录的完整语义仍由 NET08 负责；本册只关心 DNS 在完整请求中输出什么，以及下一步怎样接到 IP 转发。

7 核心训练单元：每出现一条 IP 数据报，就跑一次逐跳子程序

从这里开始，不再为 DNS、TCP 和 HTTP 各背一套“底层流程”。只要题目说“发送这条报文”，先看这条报文形成的 IP 数据报，然后执行同一个逐跳检查：

逐跳子程序

1. 写出这条数据报的 **destination IP**。DNS query 的 destination 是递归 DNS 服务器；TCP SYN 和后续 HTTP 数据的 destination 才是 Web server。
2. 查当前节点的 **FIB**。用 LPM 得到出接口和 next-hop IP；若目的位于直连网段，next hop 就是目的主机本身。
3. 准备当前这一跳。Ethernet/WLAN 若缺少下一跳的链路地址，再查 neighbor cache 并按需要执行 ARP/ND；Native PPP 只有一个对端，不做 Ethernet 式 MAC 查询。
4. 按当前链路封装并发送。这一跳使用 802.11、Ethernet、PPP 或题设给出的其他链路表示。
5. 如果下一节点是路由器，结束旧的链路帧。IPv4 路由器处理 TTL 和首部，再从“查自己的 FIB”重新开始；不要把上一跳 MAC 带到下一跳。
6. 如果已经到达目的主机，才向 TCP/UDP 交付。随后由对应的 TCP/UDP 模块继续处理端口、序号、ACK 或应用数据。

这个子程序不是说每条链路做完全相同的事，而是说**每一跳都要重新回答同一组问题：最终目的 IP 是谁、当前 FIB 让我从哪里出去、这一跳先交给谁、当前链路怎样把数据送过去**。

下面选一条从客户端发往 Web 服务器的 IPv4 数据报，按设备顺序把这个子程序完整跑一遍。DNS 查询、TCP SYN、TCP ACK 和承载 HTTP 数据的报文段都会重复调用它；只是它们的目的 IP、方向和上层语义可能不同。

7.1 客户端：先判断目标是不是本地，再确定下一跳

目标地址为 203.0.113.80。客户端根据自己的前缀判断它不在 192.168.10.0/24 内，于是查 FIB 命中默认路由：

destination IP = 203.0.113.80, next-hop IP = 192.168.10.1.

这两个“目的”必须同时保留：IP 层描述的最终目的仍是 Web 服务器；当前链路只负责先把帧交给 R1。

如果 ARP cache 中没有 R1 的 MAC，客户端就在 VLAN 10 的当前广播域内发送 ARP Request。ARP Request 不会穿过 R1 去询问远端 Web server。

7.2 STA 到 AP：802.11 只解决这一段无线链路

经典 infrastructure BSS 中，STA 向 distribution system 发送数据时 To DS=1, From DS=0：

- Address 1: RA/BSSID，即当前无线接收端 AP；
- Address 2: TA/SA，即发送 STA；
- Address 3: distribution system 一侧的最终二层目的，本例为 R1 的 LAN MAC；

- Address 4: 这一方向不使用。只有 To DS=1, From DS=1 的无线桥接/WDS 等场景才需要第四地址。

CSMA/CA、DIFS/SIFS、随机退避、NAV、RTS/CTS 和 MAC ACK 都只服务当前 WLAN hop。收到 802.11 ACK 只能说明这一跳得到链路层反馈，不能推出远端 HTTP 已成功。

7.3 AP 到交换网络：换成 Ethernet 表示，但还没有经过 IP router

AP 在主场景中是 bridge。它把 802.11 数据帧中的上层 packet 转为 Ethernet 帧继续送入 VLAN 10:

- Ethernet source MAC 仍对应 STA;
- Ethernet destination MAC 为当前三层下一跳 R1;
- 无标签 Ethernet II 中 IPv4 EtherType 为 0x0800;
- 若在 trunk 上使用 802.1Q, 则会插入 VLAN tag, 常见 TPID 为 0x8100;
- FCS 属于当前帧的检错信息, 不是端到端校验。

AP bridge 和二层交换机都不是 IP router, 因此不会因为经过它们就让 IPv4 TTL 减 1。

7.4 Switch: 先学习 source, 再根据 destination 决定怎么转发

交换机收到 frame 后, 先用 source MAC 更新 MAC table, 再查 destination MAC:

1. destination 未知: 在同 VLAN、且 STP 允许转发的其他端口上泛洪;
2. destination 已知且在另一个端口: 定向转发;
3. destination 已知且就在入端口: 过滤, 不再发回该端口;
4. broadcast/multicast: 按相应二层规则处理。

STP 解决的是冗余二层链路产生环路的问题。它先形成一套 active topology; 普通 frame 到来时只使用当前端口状态, 不是每个 frame 都重新运行一次 STP。

7.5 R1: 这里才真正进入下一次 IP hop

R1 接收发给自己 LAN MAC 的 Ethernet frame, 验证并结束当前链路封装, 然后处理 IPv4 packet:

1. 检查并更新 IPv4 首部相关状态;
2. TTL 减 1; 若结果为 0, 丢弃原 packet 并产生 ICMP Time Exceeded;
3. 对 destination IP 做 LPM, 决定新的出接口和下一跳;
4. 按新的出接口建立下一段链路封装。

本例 R1 还执行 NAT。私网端点

192.168.10.23:p

可映射为公网端点

198.51.100.9:p'.

因此外部看到的 source IP 和 source port 都可能变化。IPv4 首部校验和必须相应更新；TCP/UDP 校验和还覆盖 pseudo-header 中的 IP 信息，所以受影响的传输层校验和也要调整。回程数据必须命中反向 NAT mapping 才能回到内部 client。

NAPT 只负责地址/端口映射，并不负责 TCP 的可靠传输。Seq/Ack、重传和窗口状态仍由两端的 TCP 模块维护。

7.6 R1 到 R2: Native PPP 与 PPPoE 不能混在一起

R1-R2 是 Native PPP 点对点链路。这里只有一个 peer，所以不需要 Ethernet 式目的/源 MAC 学习。若题目继续追问 PPP 帧，经典 HDLC-like PPP 帧依次包含 Flag、Address、Control、Protocol、Information、FCS 和尾部 Flag。Address/Control 不是 Ethernet 的源/目的 MAC；异步链路常用字节转义，比特同步链路使用零比特填充。LCP 用于建立和配置 PPP 链路，NCP 再配置对应网络层协议。经典 PPP 可以检错并丢弃坏帧，但不因此拥有 TCP 式序号、ACK 和可靠重传。

PPPoE 则不同：它把 PPP 放在 Ethernet 上承载，所以外层仍然存在 Ethernet MAC。看到 PPPoE 不能写成“PPP 所以这段没有 MAC”。

7.7 中间路由器：重复同一个动作，不为当前 packet 临时跑一次 OSPF/BGP

R2、Core、BR1、BR2、Rn 都只对当前 packet 使用自己已经安装的转发表：

(destination IP, current FIB) ↦ (出接口, 下一跳).

每经过一个真正的 IP router，TTL 再减 1，并按新出接口重新准备本跳链路交付。OSPF/BGP 的职责是更慢时间尺度上的路由信息交换、路径选择和 FIB 更新，不是 packet 到来以后才开始计算。

7.8 最后一跳：这时 next hop 才等于最终 Web server

Rn 的 FIB 命中直连前缀 203.0.113.0/24，因此：

next-hop IP = destination IP = 203.0.113.80.

如果 ARP cache 未命中，Rn 对 Web server 做 ARP，得到 Web MAC，再创建最后一跳 Ethernet frame。服务器网卡接收后，IPv4 把数据交给相应的 TCP/UDP 模块。

8 把字段放在正确的作用范围里：哪些保持，哪些会变

前面的逐设备解释很长，可以压成一张字段与状态演进全景图（如图 4 所示）。读图时不要用一句“包一路不变”或“每跳全都变”概括，因为不同层的字段寿命并不一样。

对象 / 字段	主要作用范围	主场景中怎样变化
HTTP 资源 / 资源表示	应用层两端	中间交换机和路由器不解释 HTTP 资源语义
TCP Seq/Ack	TCP 两端	普通路由器和基础 NAPT 不维护 TCP 可靠传输状态
TCP/UDP 端口	主机内的传输端点	NAPT 可以改写公网侧看到的端口

IPv4 目的地址	跨多个路由跳	主场景中始终指向 Web server；若题设加入 DNAT/隧道再另行判断
IPv4 源地址	跨多个路由跳	在 R1 的 NAPT 处从私网地址改成公网地址
TTL	每个 IP 路由跳	只在 IP 路由器处递减；AP/交换机不消耗 TTL
MAC / 802.11 地址	当前二层交付范围	AP 可在 802.11 与 Ethernet 之间转换表示；经过路由器后按下一链路重新确定
FCS	当前链路帧	每个新帧独立计算和检查，不是端到端状态
物理信号	当前传输介质	WLAN、电缆、光纤可以使用完全不同的信号表示

一个重要边界是：**交换机内部的转发和路由器的逐跳转发不是同一件事**。在同一 Ethernet 二层交付范围内，交换机通常保持源/目的 MAC 的语义；经过 IP router 后，入链路的二层目的已经完成任务，下一条链路再按新的 next hop 建立新的封装。

9 TCP：把多条 IP 数据报组织成端到端字节流

主场景中的 HTTP 使用 TCP。TCP 连接真正由两端 TCP 栈维护，中间普通路由器不保存它的 Seq/Ack/window 状态。

9.1 端口和连接标识

端口解决的是“这台主机上的哪一个进程/传输端点应该接收数据”。IANA 常把端口范围分为 System Ports 0--1023、User Ports 1024--49151 和 Dynamic/Private Ports 49152--65535；端口 0 保留。

网络题中常用五元组

(protocol, local IP, local port, remote IP, remote port)

描述一条传输流。对已经建立的 TCP 连接，协议已经固定后，四个端点字段足以区分连接。不要把“5-tuple 唯一定位所有 socket”当成无条件定理：监听 socket 和 UDP socket 的分用条件并不完全相同。

9.2 三次握手和连接释放

经典三次握手可以写成：

$$\text{SYN}(x) \rightarrow \text{SYN} + \text{ACK}(y, x + 1) \rightarrow \text{ACK}(x + 1, y + 1).$$

每一个箭头都代表一条需要经过前述逐跳网络的 TCP segment / IP datagram。

连接关闭时，FIN/ACK、半关闭、TIME_WAIT 与异常 RST 都属于 NET06 的 TCP 生命周期。本册主线停在收到目标 HTTP 内容，因此不在这里展开完整挥手状态机；题目若继续追问连接怎样结束，再转回 NET06。

9.3 可靠性、流量控制和拥塞控制不能混成一个窗口

TCP sender 同时受到两类不同限制：

- receiver 用 rwnd 告诉 sender 自己还能接收多少；
- sender 用 cwnd 控制自己愿意向共享网络放入多少未确认数据。

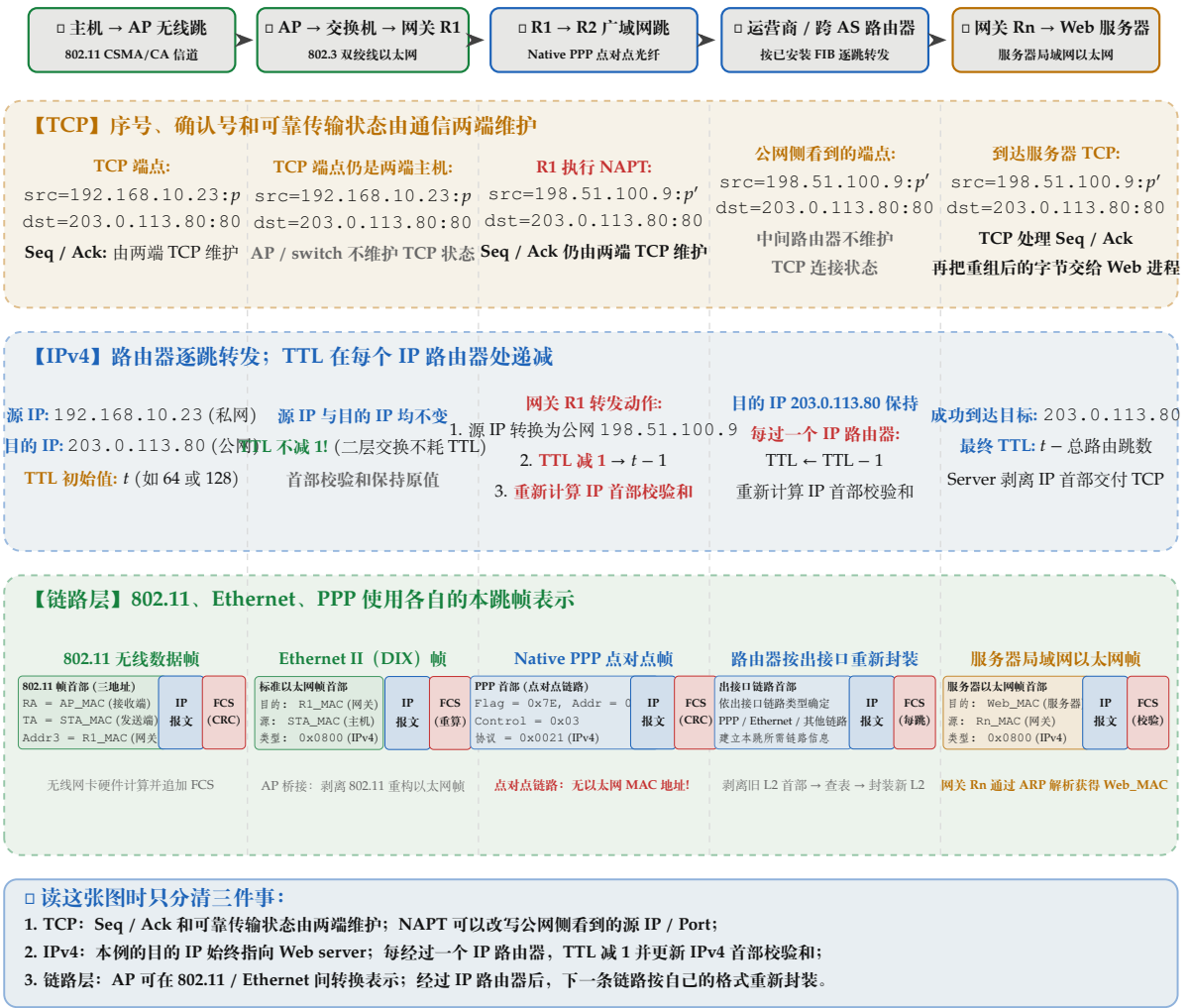


图 4 跨介质全链路帧转换与端到端状态演进。TCP 状态由两端维护, IP 首部由路由器递减 TTL, 链路层帧在每一跳按当前介质重新封装。

若 FlightSize 表示已经发出但尚未确认的数据量, 则当前还能新增发送的上界为

$$W_{\text{usable}} = \max(0, \min(rwnd, cwnd) - \text{FlightSize}).$$

rwnd=0 说明接收端暂时没有空间, sender 需要使用零窗口探测等机制等待窗口重新打开; loss、dupACK、RTO 或 ECN 等反馈则可能改变拥塞控制状态。慢开始、拥塞避免、快重传和快恢复由 NET07 完整拥有。

GBN/SR 的窗口、序号空间和回退规则由 NET03 解释; TCP 是更复杂的端到端实例。BDP 则回答“链路中要有多少在途数据才能把带宽充分利用”, 通过 NET-B05 与 window 联系起来。

9.4 UDP 在这条主线里的位置

DNS 常用 UDP; UDP header 固定包含 source port、destination port、length 和 checksum。其 checksum 计算还会使用包含源/目的 IP、协议号和长度等信息的 pseudo-header, 所以 NAT 改写地址或端口时不能只改表面字段而不处理校验和。

UDP 不建立 TCP 那样的连接状态, 也不提供 TCP 式可靠字节流。若题目比较 TCP 与 UDP, 应回到 NET06, 而不是从“是否有端口”判断是否面向连接。

10 HTTP：字节到了以后，应用层才重新解释“资源”

TCP 只提供字节流，不知道这些字节代表 HTML、图片还是状态码。HTTP 才解释 `method`、`target`、`header fields`、`status` 和 `resource semantics`。

10.1 资源和资源表示要分开

Resource（资源） 是 URI 所标识的逻辑对象；**representation（资源表示）** 是服务器在某次响应中用来表示这个资源的具体内容和元数据。同一个资源可以有不同表示，因此不能把“资源”直接等同于某一份固定字节文件。

常见 `method`（方法）如 `GET`、`HEAD`、`POST`、`PUT`、`DELETE`、`OPTIONS`；状态码按 `1xx-5xx` 分组。本册不重新列完整状态码表，只强调它们属于应用层语义，不能拿 HTTP 状态码替代 TCP/IP 层的成功证据。

10.2 一个网页往往不是一个对象

主 HTML 返回后，浏览器才可能从中发现 CSS、JavaScript、图片等 URL。因此页面加载更像一个依赖图：

main HTML → parse → {CSS, JS, images, ...}.

这就是为什么性能题不能只写“HTTP 需要 1 RTT”。必须先看哪些对象必须串行发现，哪些连接能复用，哪些请求可以并行或多路复用（`multiplex`）。

10.3 HTTP cache 会改变整个网络事件图

如果本地缓存仍然新鲜，并且足以直接满足当前请求，后续网络交互可以整体消失；若缓存需要重新验证，浏览器可以发送条件请求。服务器返回 304 时，表示客户端可以继续使用本地已有的资源表示，而不是“这个请求叫 304”。

10.4 HTTP 版本改变的是连接与并发组织方式

版本	典型考试观察点	不能误读为
HTTP/1.0	默认非持久连接（ <code>non-persistent</code> ）时，多个对象常反复建立 TCP 连接	每个页面永远只有一条连接
HTTP/1.1	默认持久连接（ <code>persistent</code> ）；是否允许 <code>pipeline</code> 取决于题设	持久连接不等于所有对象自动并行
HTTP/2	多个 <code>stream</code> 可复用一条 TCP <code>connection</code>	TCP 丢包仍可能使多个 <code>stream</code> 一起等待底层有序字节流
HTTP/3	基于 QUIC 的独立 <code>stream</code> 减少共享 TCP 字节流造成的跨 <code>stream</code> 阻塞	单个 <code>stream</code> 内仍要求可靠、有序交付

11 路由协议在后台工作，当前数据报只看当前 FIB

路由与转发必须分开：

路由协议负责形成/更新转发表

当前数据报只使用已经生效的转发表

机制	它在后台做什么	当前数据报真正使用什么
STP	在冗余二层拓扑中形成无环的 active topology	switch 当前端口状态
Switch learning	从 source MAC 学习 MAC-port mapping	destination MAC lookup
RIP	距离向量交换, 经典 metric 为 hop count, 存在 count-to-infinity 等边界	安装后的 FIB
OSPF	邻接、LSA、LSDB、SPF 计算域内路由	安装后的 FIB
BGP	AS 间交换 prefix 和 path attributes, 并按 policy 选择路径	border router 当前已安装的下一跳/出接口

特别注意时间顺序: 收到 routing update, 不等于新 route 已经 selected; selected route 不等于 FIB 已安装; FIB 已安装也不能反过来改变更早到达的数据报走过的路径。

12 三类异常不要混在一起: 生命周期、尺寸和反馈

12.1 TTL / ICMP: 数据报还能不能继续往前走

每经过一个 IP router, IPv4 TTL 减 1。减到 0 的 router 丢弃原 packet, 并产生 ICMP Time Exceeded。ICMP Echo Request/Reply 则可以用于主动探测 IP 层可达性。

ICMP 是网络层反馈, 不等于 TCP ACK, 也不等于“网页一定可以正常访问”。

12.2 MTU: 下一条链路能不能装下这个 IP packet

链路 MTU 会向上限制 IP packet 的大小:

Link MTU → IP packet size → fragmentation / PMTU → TCP MSS / segment size.

主场景故意让 PPP WAN 的 MTU 为 800 B。IPv4 packet 超过出接口 MTU 时, 是否允许分片要看 DF 和题设; fragment offset、MF 和目的端重组由 NET04 负责。若 TCP 已根据 MSS/PMTU 把 segment 控制在更合适的大小, 可以避免不必要的 IP fragmentation。

12.3 链路 ACK、TCP ACK、rwnd、cwnd、ICMP、HTTP status 各自说明不同事情

看到什么反馈	它主要说明什么	不能直接推出什么
802.11 MAC ACK	当前无线单跳得到链路层确认	Web server 已收到 HTTP
TCP ACK / dupACK / RTO	TCP byte delivery 的端到端证据与恢复触发	接收缓冲区或路由状态本身
rwnd	receiver 还能接收多少	网络路径是否拥塞
cwnd 的变化	sender 对共享路径拥塞程度的控制结果	HTTP cache 是否命中
ICMP error	IP 层路径/packet 处理出现特定问题	TCP 已正常关闭
HTTP status	application 对 request 的语义响应	下层从未发生丢包/重传

13 性能题先画依赖关系，再算 RTT 和传输时间

网络时延的基本账仍来自 NET01:

$$d_{total} = d_{proc} + d_{queue} + d_{trans} + d_{prop}, \quad d_{trans} = \frac{L}{R}.$$

传播时延由距离和传播速度决定，发送时延由数据量和链路速率决定，二者不能互换。BDP

$$BDP = \text{bandwidth} \times RTT$$

表示路径上为了充分利用带宽，大致需要容纳多少在途数据。

对于 Web 访问，先画真正的依赖关系：

network configuration if needed → DNS if needed → TCP setup if needed → main HTTP object → subresource d

然后再删掉已经命中的状态，标出能并行的部分，最后才相加 RTT 和 transmission time。不能机械套 “DNS 1 RTT + TCP 1 RTT + HTTP 1 RTT”。

14 如果把主场景的一段网络换掉，应该切换到什么机制

主场景不需要把所有协议同时运行，但考试可能把其中某一段换掉。此时只替换对应机制，不要让无关层一起变化。

题设怎样改	应切换到什么机制	仍然保持什么
switched full-duplex Ethernet 改为共享半双工 Ethernet	CSMA/CD、2τ、64 B 最小帧与二进制指数退避的经典模型	IP destination、TCP 端点语义不变
WLAN 出现 hidden station	RTS/CTS、NAV、SIFS/DIFS 更重要	仍只解决当前 WLAN hop 的介质访问
随机竞争改成 polling / token	发送权由轮询或令牌控制，会引入等待与控制开销	不取代 IP routing
共享资源改为 FDM/TDM/CDM	静态划分频率、时间或正交码；CDM 用码向量相关/内积区分用户	不取代链路以上协议
Native PPP 改为 PPPoE	PPP 外面再套 Ethernet，MAC 地址重新出现	PPP 会话/控制语义仍存在
IPv4 改为 IPv6	SLAAC/RA、ND、Hop Limit 等替换 IPv4 对应机制；router 不做 IPv4 式中途分片	destination 与 next hop 仍要分清
OSPF 改为 RIP	控制面的信息交换、metric 和收敛行为改变	当前 packet 仍使用已安装 FIB
HTTP/1.x 改为 HTTP/2/3	连接与 stream 的并发组织、队头阻塞边界改变	资源/资源表示仍属于应用层

802.1Q 最小帧长度、不同标准/实现的 padding 口径可能不同，涉及数值时必须按题目给定的帧长定义计算，不把 “带 VLAN 一定是 68 B” 当成无条件规则。

15 故障题：只追到第一处真正断掉的地方

看到“网页打不开”“超时”“DNS 成功但 HTTP 没返回”时，不要直接猜自己最熟悉的协议。按照实际过程检查：

1. 当前请求是不是已经被本地 HTTP cache 满足？如果没有，主机网络配置是否有效？
2. 若 URL 使用域名，目标 IP 是否已经得到？
3. 准备发送的应用数据是否已经形成合法的 TCP/UDP 报文和 IP 数据报？
4. 当前设备的 FIB 是否给出可执行的出接口和下一跳？
5. 当前链路需要的下一跳信息是否存在？例如 Ethernet 的 ARP/ND 状态、PPP 的对端状态。
6. 当前链路帧能否在 VLAN、STP 和 MAC 表的当前状态下完成这一跳？
7. 经过路由器时，TTL、MTU、NAT 和 LPM 的处理结果是否允许继续转发？
8. TCP/UDP 以及最终 HTTP 语义是否继续成立？

第一个有证据“不成立”的位置就是**第一次断点 (First Divergence)**。后面的现象可能只是它的连锁结果，不要越过第一处断点继续猜。

16 贯穿母例：冷启动、热访问和只改一个条件

16.1 冷启动：几乎没有可复用状态

设：DHCP lease 不存在、DNS 缓存未命中、默认网关 ARP 表项不存在、交换机 MAC 表初始为空、TCP 连接不存在、HTTP 缓存未命中，但 STP 和路由已经收敛到可用状态。实际过程可以写成：

DHCP 建立主机网络配置

- > 得到 IP / 前缀 / 默认网关 / DNS 服务器
- > 需要 DNS 查询
 - > FIB 先选到 DNS server 的下一跳
 - > 需要时 ARP 默认网关
 - > WLAN / Ethernet / routers 把 DNS query 送到 resolver
- > 得到 Web server IP
- > 建立 TCP 连接
 - > 网关 ARP 表项仍有效就直接复用
 - > SYN / SYN+ACK / ACK 每一条都重新走逐跳交付
 - > R1 建立或复用 NAT 映射
- > HTTP request / response
- > TCP 重组并交付字节
- > HTTP 解释资源表示
- > HTML 还可能引出 CSS / JS / 图片等新请求

最值得注意的是不是步骤很多，而是：**ARP 可以先为了 DNS query 建立，随后继续被 TCP/HTTP 数据复用**。所以 ARP 绝不能固定画在 DNS 之后。

16.2 热访问：只省掉仍然有效的部分

第二次很快访问同一站点时，DHCP 租约、DNS 记录、默认网关 ARP 表项、MAC 表、NAT 映射、TCP 连接或 HTTP 缓存可能仍有部分有效。

- DNS 缓存命中：不重新发 DNS 查询，但后续 HTTP 数据仍要正常转发；
- ARP 缓存命中：不重新发本链路的 ARP Request，但最终 destination IP 不变；
- TCP 连接仍可复用：不重新三次握手，但仍要发送需要的 HTTP 请求；
- HTTP 缓存仍新鲜并足以满足请求：本次访问可能直接在本地结束，后续网络事件整体消失。

16.3 只改一个条件：只允许相关子过程变化

这是检验模型是否真正分层的最好方法之一。

只改什么	应改变什么	不应该跟着改变什么
DNS 未命中 → 命中	删除 DNS 查询过程	后续 Web 数据报仍要查 FIB 并完成链路交付
邻居缓存未命中 → 命中	删除当前链路的 ARP/ND 交换	IP 目的地址、TCP 连接状态
TCP 不存在 → 可复用	删除三次握手	HTTP 请求语义
FIB 旧版本 → 新版本	新版本生效后到达的数据报使用新的转发动作	已缓存的 DNS 记录
MTU 1500 → 800	改变分片、MSS 和每个报文大小相关分支	域名本身
HTTP 缓存未命中 → 本地缓存可直接使用	后续网络交互可能整体消失	后台已经形成的路由/STP 状态

17 考纲覆盖检查：没有塞进 Web 主线的知识放在哪里

主线只选择一组可以同时成立的机制。下面这张表的作用是防止“没有画进主线 = 被删除”的误解。

知识点	在 NET-I01 中怎样出现	完整机制见
Ethernet / VLAN / 802.11 / PPP framing、FCS、透明传输	逐跳路径和字段变化图只保留必要接口	NET02
CSMA/CD、CSMA/CA、WLAN 主线 + 替换场景 polling/token、FDM/TDM/CDM		NET01 / NET02
Switch learning、unknown-unicast flooding、filtering、STP	接入 LAN 主线	NET02

DHCP DORA/Relay; IPv6 SLAAC/ND	启动阶段主线 + IPv6 替换场景	NET04
same-subnet 判断、 CIDR/LPM、默认路由、 ARP	每条数据报的逐跳交付过程	NET04; 接口见 NET-B01
NAPT、TTL、MTU、IPv4 fragmentation、ICMP	R1 边界与异常分支	NET04; MTU 接口见 NET-B06
RIP、OSPF、BGP、收敛 中间态	OSPF/BGP 主场景背景状态; RIP 替换场 景	NET05; 接口见 NET-B02
端口、UDP header / pseudo-header checksum	DNS/传输层接口	NET06
TCP 三次握手、连接释放、 RST、rwnd、零窗口	三次握手进入主线; 连接关闭在本册停止 位置之后	NET06
Stop-and-Wait、GBN、 SR、序号空间与窗口	作为理解 TCP 可靠传输的前置模型, 本册 不重复展开	NET03 / NET-B03
slow start、congestion avoidance、fast retrans- mit/recovery、cwnd	TCP 反馈与可用窗口	NET07; rwnd/cwnd 接口见 NET-B04
BDP 与 window	性能推导	NET01 / NET-B05
DNS hierarchy、RR、cache TTL	名字解析主线	NET08
HTTP 资源/资源表示、 method/status、cache、 对象依赖、HTTP/1.x-3	应用阶段与性能题	NET08
FTP control/data connec- tions	Web 主线之外的应用协议替换场景	NET08
SMTP/POP3/IMAP/MIME 邮件应用协议, 不属于 Web 主线		NET08

18 做综合题时的最短落笔顺序

考场不需要复述整本手册。保留下面六步即可:

1. **先写目标和已有状态:** URL/目标是什么? 配置、DNS、ARP、FIB、TCP、HTTP cache 哪些已经存在?
2. **先判断当前还缺什么:** 不要把所有协议机械重跑一遍。
3. **一旦要发送一条普通 IP 数据报:** 查目的 IP → 当前 FIB → 出接口/下一跳 → 准备本跳链路交付 → 下一节点。
4. **每到一个设备只让它做自己的事:** 交换机不减 TTL, 路由器不解释 HTTP, DNS 不输出 MAC, TCP 模块不生成路由表。
5. **性能题先画依赖关系, 故障题先找第一次断点。**

6. **最后反查字段**：destination IP、next-hop IP、MAC、port、TTL 有没有被混成同一种“地址”或同一种状态？

19 独立检查：不用原推理再算一遍

推演结束后至少做六个反向检查：

1. **名字类型**：domain、destination IP、next-hop IP、MAC、port 有没有被混成同一种“地址”？
2. **IP 跳数**：是不是只有真正的 IP 路由器才让 TTL/Hop Limit 推进？
3. **链路帧**：到了路由器或明确的链路表示转换边界后，旧二层表示是否还被错误沿用？
4. **状态归属**：MAC 表、FIB、ARP cache、TCP state、DNS cache 是否由正确设备或协议模块维护？
5. **时间先后**：OSPF/BGP/STP 形成状态、状态生效、当前数据报使用状态，有没有被写成同一时刻？
6. **性能代价**：RTT、发送时延和传播时延是否来自真实依赖关系，而不是按协议名机械加常数？

本册在哪里停止

本册到“目标 HTTP 响应的资源表示已经被应用层解释”为止。TCP 四次挥手、半关闭、TIME_WAIT 和 RST 的完整状态机仍由 NET06 负责；TLS 详细握手、QUIC 内部恢复、CDN 选站、浏览器渲染和内核/NIC 内部执行路径属于其他专题。

停止在这里不是删除这些知识，而是保持职责清楚：NET-I01 负责把一次 Web 请求串起来，不再复制每个局部协议的完整教材。

最短压缩

一次网络请求可以记成两句话：

先看这次访问还缺什么状态，只补缺失的部分；一旦要发送一条普通 IP 数据报，就用当前设备的 FIB 找出下一跳，并按当前链路把它送过去。

经过 router 时，入链路帧结束、TTL 推进、下一跳重新决定；经过 AP/switch 时不要误加 IP hop。所有综合题的关键，就是始终分清“最终目的”和“当前下一跳”，分清“哪台设备保存什么状态”，并把“路由/STP 状态怎样形成”与“当前数据报怎样使用这些状态”分开。